
title: Amazon S3 Bucket Setup Guide subtitle: Reusable guide for any Node.js / Express project

Amazon S3 Bucket Setup Guide

Purpose: Step-by-step instructions to create and configure a private S3 bucket, IAM credentials, and environment variables so any backend (Express, NestJS, etc.) can upload files securely using AWS SDK v3 and presigned URLs.

Estimated time: 30–45 minutes (first time)

Table of Contents

1. [Prerequisites](#)
 2. [Create an S3 Bucket](#)
 3. [Keep the Bucket Private](#)
 4. [Create an IAM User for Programmatic Access](#)
 5. [Attach the Minimum IAM Policy](#)
 6. [Optional — Bucket CORS \(Direct Browser Upload\)](#)
 7. [Collect Your Configuration Values](#)
 8. [Configure Your Backend \(.env\)](#)
 9. [Verify Access from Your Machine](#)
 10. [Integrate in Node.js \(AWS SDK v3\)](#)
 11. [Production Best Practices](#)
 12. [Troubleshooting](#)
 13. [Checklist for New Projects](#)
-

1. Prerequisites

- An **AWS account** (<https://aws.amazon.com/>)
- Permission to create S3 buckets and IAM users (or ask your admin)
- A backend project where you will store **only object keys** in the database (not public URLs), and serve files via **presigned URLs**

You will need these four values for every project:

Variable	Example	Description
<code>AWS_ACCESS_KEY_ID</code>	<code>AKIA...</code>	IAM access key
<code>AWS_SECRET_ACCESS_KEY</code>	<code>wJalr...</code>	IAM secret (shown once)
<code>AWS_REGION</code>	<code>ap-south-1</code>	Region where bucket was created
<code>AWS_BUCKET_NAME</code>	<code>my-app-uploads-prod</code>	Globally unique bucket name

2. Create an S3 Bucket

1. Sign in to **AWS Console** → search **S3** → **Create bucket**.
2. **Bucket name:** Choose a unique name (e.g. `my-company-app-uploads-prod`). Names are global across all AWS accounts.
3. **AWS Region:** Pick the region closest to your users or your server (e.g. `Asia Pacific (Mumbai) ap-south-1`). **Remember this region** — it becomes `AWS_REGION`.
4. **Object Ownership:** ACLs disabled (recommended) is fine.
5. **Block Public Access:** Leave **all four options enabled** (recommended). Your app will use IAM + presigned URLs, not public objects.
6. **Bucket Versioning:** Optional (useful for recovery; slightly higher cost).
7. **Default encryption:** Enable **SSE-S3** or **SSE-KMS** (recommended).
8. Click **Create bucket**.

3. Keep the Bucket Private

Confirm after creation:

- **Block all public access** = On
- No bucket policy that grants `Principal: "*" with s3:GetObject`
- No static website hosting unless you intentionally want a public site (not needed for private uploads)

Why: Private bucket + presigned URLs = files are not guessable or permanently public.

4. Create an IAM User for Programmatic Access

Do **not** use your root AWS account keys in application code.

1. AWS Console → **IAM** → **Users** → **Create user**.

2. **User name:** e.g. `my-app-s3-uploader`.
3. **Provide user access to AWS console:** No (access key only).
4. **Attach policies directly:** Skip for now; use inline policy in next step.
5. Create user → open user → **Security credentials** → **Create access key**.
6. **Use case:** Application running outside AWS (or local dev).
7. Save **Access key ID** and **Secret access key** immediately. The secret is shown **only once**.

These map to:

- `AWS_ACCESS_KEY_ID`
- `AWS_SECRET_ACCESS_KEY`

5. Attach the Minimum IAM Policy

Replace `YOUR_BUCKET_NAME` with your actual bucket name.

Recommended policy (upload, read, delete for one bucket):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ListBucket",
      "Effect": "Allow",
      "Action": ["s3:ListBucket"],
      "Resource": "arn:aws:s3:::YOUR_BUCKET_NAME"
    },
    {
      "Sid": "ObjectAccess",
      "Effect": "Allow",
      "Action": [
        "s3:PutObject",
        "s3:GetObject",
        "s3:DeleteObject",
        "s3:HeadObject"
      ],
      "Resource": "arn:aws:s3:::YOUR_BUCKET_NAME/*"
    }
  ]
}
```

How to attach:

1. IAM → Users → your user → **Add permissions** → **Create inline policy** → JSON → paste → **Review** → name e.g. `S3AppBucketAccess` → **Create**.

Optional stricter policy: Restrict `s3:PutObject` to prefix `uploads/*` only:

```
"Resource": "arn:aws:s3:::YOUR_BUCKET_NAME/uploads/*"
```

6. Optional — Bucket CORS (Direct Browser Upload)

Only needed if the **browser** uploads directly to S3 using presigned PUT URLs (not through your API).

1. S3 → your bucket → **Permissions** → **Cross-origin resource sharing (CORS)** → Edit:

```
[
  {
    "AllowedHeaders": ["*"],
    "AllowedMethods": ["GET", "PUT", "POST", "HEAD"],
    "AllowedOrigins": [
      "http://localhost:3000",
      "https://your-frontend.vercel.app"
    ],
    "ExposeHeaders": ["ETag"],
    "MaxAgeSeconds": 3000
  }
]
```

Replace origins with your real frontend URLs. If all uploads go through your backend (`multipart/form-data` to Express), CORS on the bucket is often **not** required.

7. Collect Your Configuration Values

Item	Where to find it
Bucket name	S3 → Buckets → name
Region	S3 bucket overview → AWS Region (e.g. <code>ap-south-1</code>)
Access Key ID	IAM user → Security credentials
Secret Access Key	Shown once when key was created

8. Configure Your Backend (.env)

Never commit real secrets to Git. Use `.env` locally and your host's environment UI in production.

```
AWS_ACCESS_KEY_ID=AKIAxxxxxxxxxxxxxxxxxx
AWS_SECRET_ACCESS_KEY=your-secret-key-here
AWS_REGION=ap-south-1
AWS_BUCKET_NAME=your-bucket-name
```

Add `.env` to `.gitignore`. Provide the same variables in:

- Vercel → Project → Settings → Environment Variables
- Railway / Render / EC2 / Docker secrets

9. Verify Access from Your Machine

Install AWS CLI (optional): <https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html>

```
aws configure
# Enter Access Key, Secret, Region

aws s3 ls s3://YOUR_BUCKET_NAME/
aws s3 cp ./test.png s3://YOUR_BUCKET_NAME/uploads/test.png
aws s3 rm s3://YOUR_BUCKET_NAME/uploads/test.png
```

If these succeed, IAM and bucket name are correct.

Quick Node test (optional):

```
npm install @aws-sdk/client-s3
```

```
import { S3Client, HeadBucketCommand } from "@aws-sdk/client-s3";

const client = new S3Client({
  region: process.env.AWS_REGION,
  credentials: {
    accessKeyId: process.env.AWS_ACCESS_KEY_ID,
    secretAccessKey: process.env.AWS_SECRET_ACCESS_KEY
  }
});

await client.send(new HeadBucketCommand({ Bucket: process.env.AWS_BUCKET_NAME }));
console.log("Bucket accessible");
```

10. Integrate in Node.js (AWS SDK v3)

Install:

```
npm install @aws-sdk/client-s3 @aws-sdk/s3-request-presigner
```

Typical architecture:

1. Client sends file to your API (`multipart/form-data`).
2. API uploads to S3 with `PutObjectCommand` .
3. API saves **object key** in DB (e.g. `uploads/1717344000000-uuid-photo.png`).
4. When UI needs to display the file, API returns a **presigned GET URL** (expires in 15–60 minutes).

Object key naming (recommended):

```
uploads/{timestamp}-{uuid}-{sanitized-original-filename}
```

Environment variables in code: Read from `process.env` only — never hardcode keys in source files.

11. Production Best Practices

Practice	Why
Private bucket	Prevents anonymous downloads
Presigned URLs	Time-limited access without making objects public

Practice	Why
Store keys in DB, not permanent URLs	URLs expire; keys are stable
IAM user per app / environment	Limit blast radius
Separate buckets for dev/staging/prod	Avoid accidental data mix
Rotate access keys periodically	Security hygiene
Enable encryption at rest	Default SSE-S3 on bucket
Validate file type and size in API	Prevent abuse
Use least-privilege IAM policy	Only required actions on one bucket

12. Troubleshooting

Error / symptom	Likely cause	Fix
<code>AccessDenied</code>	IAM policy missing action or wrong bucket ARN	Update inline policy; check bucket name
<code>NoSuchBucket</code>	Wrong <code>AWS_BUCKET_NAME</code> or wrong region	Match bucket name and <code>AWS_REGION</code>
<code>InvalidAccessKeyId</code>	Wrong or deleted access key	Create new key; update env
<code>SignatureDoesNotMatch</code>	Wrong secret key	Re-copy secret; no extra spaces in <code>.env</code>
Missing required environment variable	Env not loaded in production	Set vars on Vercel/host; redeploy
Upload works locally, fails on Vercel	Env vars not set on Vercel	Add all four <code>AWS_*</code> variables for Production
Images break after 15 minutes	Presigned URL expired	Regenerate URL on each API response
CORS error in browser direct upload	Bucket CORS or wrong origin	Update S3 CORS <code>AllowedOrigins</code>

13. Checklist for New Projects

- ☐ S3 bucket created in chosen region
 - ☐ Block public access enabled
 - ☐ Encryption enabled
 - ☐ IAM user created (no console access)
 - ☐ Inline policy attached (single bucket)
 - ☐ Access key saved securely
 - ☐ Four env vars set locally and on host
 - ☐ `HeadBucket` or CLI `aws s3 ls` succeeds
 - ☐ Backend uploads and returns presigned download URL
 - ☐ Database stores object keys only
 - ☐ `.env` in `.gitignore`
 - ☐ (Optional) CORS configured for direct browser upload
-

Document version: 1.0

Applies to: AWS S3 + IAM + Node.js AWS SDK v3

Related: See your project's `docs/s3-upload.md` for API endpoint details after integration.